

MSc Applied AI - WMG

AIDL Local Machine Setup Guide

Contents

1. Goal	2
2. Folder Structure.....	2
3. Before You Start	2
4. Create the Folders.....	3
5. Put the Requirements Files in the Right Place	3
6. Open VS Code in the Correct Folder	3
7. Check Python	3
8. Create the Virtual Environment.....	4
9. Fix the PowerShell Activation Restriction.....	4
9.1. If Activation Still Fails.....	4
10. Always Use <code>python -m pip</code>	4
11. Upgrade Packaging Tools.....	4
12. CPU Pathway	5
13. GPU Pathway	5
14. Register the Environment for Jupyter	5
15. Tell VS Code Which Python to Use.....	6
16. How to Verify the Setup	6
17. How to Check Whether GPU Is Working	6
18. How to Run a Python File	6
19. How to Run a Notebook File	7
20. Everyday Workflow for Students.....	7
21. How to Add More Packages Later	7
22. How to Save the Installed Environment	8
23. Common Problems and Fixes	8
24. Setup Summary - Recommended Commands.....	8
CPU setup	8
GPU setup	9

1. Goal

You will create:

- **one shared virtual environment** for the whole module using VS Code
- multiple project folders for different coursework, labs, and experiments
- a **CPU pathway** for standard projects
- a **GPU pathway** for projects that need NVIDIA GPU acceleration

2. Folder Structure

You should organise their folders like this:

```
Documents/  
└─ student_id/  
    └─ venv/  
    └─ deepLearningCourse/  
        ├── requirements-cpu.txt  
        ├── requirements-gpu-cu128.txt  
        ├── requirements-gpu-rest.txt  
        ├── README_DL_ENV.md  
        └── setup_notes.txt  
    └─ Project_1/  
        ├── notebooks/  
        ├── scripts/  
        ├── data/  
        └── outputs/  
    └─ Project_2/  
        ├── notebooks/  
        ├── scripts/  
        ├── data/  
        └── outputs/  
    └─ Project_3/  
        ├── notebooks/  
        ├── scripts/  
        ├── data/  
        └── outputs/
```

What goes where

- venv/

The shared Python environment used across all projects.

- deepLearningCourse/

Store the requirements files and any setup notes here.

- Project_1/, Project_2/, etc.

Store actual coursework, notebooks, scripts, data, and outputs here.

Important rule: Do not create a separate `venv` inside each project. Create one `venv` only, at the `student_id` level.

3. Before You Start

Make sure the following are installed:

- **Python 3.13 or 3.14** `cmd` `py --list`
- **VS Code**

- VS Code extensions:
 - Python
 - Jupyter
 - Pylance
- [recommend: colab, github copilot chat](#)

PyTorch currently supports Python 3.10+, so Python 3.13 or 3.14 is acceptable for current stable releases. ([PyTorch](#))

4. Create the Folders

Open **File Explorer** and create:

```
Documents\student_id
Documents\student_id\deepLearningCourse
Documents\student_id\Project_1
Documents\student_id\Project_1\notebooks
Documents\student_id\Project_1\scripts
Documents\student_id\Project_1\data
Documents\student_id\Project_1\outputs
```

Repeat the same structure for future projects.

5. Put the Requirements Files in the Right Place

Inside: Documents\student_id\deepLearningCourse
store these files:

```
requirements-cpu.txt
requirements-gpu-cul28.txt
requirements-gpu-rest.txt
```

This separates the GPU PyTorch install from the rest of the packages, which is cleaner and avoids the CPU build being installed accidentally from the default PyPI index. PyTorch's official install page shows the CUDA 12.8 wheel index format for Windows GPU installs. ([PyTorch](#))

6. Open VS Code in the Correct Folder

In VS Code:

- Click **File**
- Click **Open Folder**
- Open: Documents\student_id

Do **not** open only Project_1 the first time.
Open the student_id root so you can create and manage the shared environment properly.

7. Check Python

[VS Code](#) [view tab -> terminal](#)

Open a terminal in VS Code and run: `python --version`

You should see something like:

```
Python 3.13.x
or
Python 3.14.x
```

If `python` does not work, try:

```
py --version
```

If `py` works but `python` does not, use `py` in the commands below instead of `python`.

8. Create the Virtual Environment

From inside: `Documents\student_id`

```
run: python -m venv venv
```

If your machine prefers the Windows launcher, use: `py -m venv venv`

This creates: `Documents\student_id\venv`

9. Fix the PowerShell Activation Restriction

The WMG -managed machines block activation scripts by default.

In the same VS Code terminal, run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

This only affects the **current terminal session**.

Then activate the `venv`:

```
.\venv\Scripts\activate
```

If activation succeeds, the terminal prompt will start with: `(venv)`

9.1. If Activation Still Fails

You can still use the environment without activation by calling its Python directly:

```
.\venv\Scripts\python.exe --version
```

This is a valid fallback.

10. Always Use `python -m pip`

On some managed machines, `pip.exe` is blocked by group policy. If are likely to face this problem. So you should **always** install packages like this:

```
python -m pip install ...
```

and **not** like this:

```
pip install ...
```

If using the fallback direct interpreter (step 9.1), use:

```
.\venv\Scripts\python.exe -m pip install ...
```

11. Upgrade Packaging Tools

Once the `venv` is active, run:

```
python -m pip install --upgrade pip setuptools wheel
```

If you are not using activation, run:

```
.\venv\Scripts\python.exe -m pip install --upgrade pip setuptools wheel
```

Now, you should choose **one** of the following:

- **CPU pathway**

- **GPU pathway**

12. CPU Pathway

Use this if:

- your project does not need GPU acceleration
- you are on a laptop without a suitable NVIDIA GPU
- you want the simplest and most robust setup

From:

Documents\student_id

run:

```
python -m pip install -r .\deepLearningCourse\requirements-cpu.txt
python -m pip install torch torchvision torchaudio
```

This installs the general package set plus standard PyTorch. If no CUDA-enabled wheel is used, PyTorch will operate in CPU mode. PyTorch's official installer distinguishes clearly between CPU and CUDA pathways. ([PyTorch](#))

13. GPU Pathway

Use this if:

- your machine has a supported NVIDIA GPU
- your project needs GPU acceleration
- you want to use CUDA with PyTorch

For **Blackwell-class NVIDIA GPUs**, CUDA 12.8 is the recommended pathway. PyTorch's official selector includes a CUDA 12.8 Windows install path, and NVIDIA's Blackwell software guidance aligns with CUDA 12.8 support. ([PyTorch](#))

Step 1 — install CUDA-enabled PyTorch

```
python -m pip install -r .\deepLearningCourse\requirements-gpu-cu128.txt
```

Step 2 — install the remaining packages

```
python -m pip install -r .\deepLearningCourse\requirements-gpu-rest.txt
```

Important note

If you previously installed a **CPU-only** PyTorch build, uninstall it first:

```
python -m pip uninstall -y torch torchvision torchaudio
```

Then reinstall using the GPU file above.

```
.\venv\Scripts\python.exe -m pip uninstall -y torch torchvision torchaudio
```

```
.\venv\Scripts\python.exe -m pip install -r .\deepLearningCourse\requirements-gpu-cu128.txt
```

14. Register the Environment for Jupyter

After installing packages, run:

```
python -m ipykernel install --user --name dl_env --display-name "Python (Deep Learning)"
```

This makes the environment appear in Jupyter and VS Code notebook kernels.

15. Tell VS Code Which Python to Use

In VS Code:

- Press `Ctrl + Shift + P`
- Search for: `Python: Select Interpreter`
- Choose: `Documents\student_id\venv\Scripts\python.exe`

This step is essential. It ensures VS Code runs scripts and notebooks using the shared environment. [double check in the terminal](#) [Get-Command python | Select-Object Source](#)

16. How to Verify the Setup

Run this command in the terminal:

```
python -c "import sys; print(sys.executable)"
```

It should show:

```
...\Documents\student_id\venv\Scripts\python.exe
```

That confirms the correct environment is being used.

17. How to Check Whether GPU Is Working

Run:

```
python -c "import torch; print(torch.__version__); print(torch.version.cuda); print(torch.cuda.is_available())"
```

Expected results

CPU pathway

You may see:

```
2.x.x+cpu
None
False
```

GPU pathway

You want something like:

```
2.x.x+cu128
12.8
True
```

[possible issue caused by unpinned requirements: pip chose newer CPU-only torch and torchvision from PyPI instead of the older CUDA 12.8 builds.](#)

You can also test in Python:

```
import torch

print("PyTorch version:", torch.__version__)
print("CUDA version:", torch.version.cuda)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))
```

18. How to Run a Python File

Suppose you create this file:

```
Documents\student_id\Project_1\scripts\test_torch.py
```

with this content:

```
import torch
```

```
print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

Run it from the terminal

```
python .\Project_1\scripts\test_torch.py
```

Or use VS Code

- Open the file
- Click **Run Python File**

If the interpreter is set correctly, it will run inside the shared `venv`.

19. How to Run a Notebook File

Create a notebook here:

```
Documents\student_id\Project_1\notebooks\demo.ipynb
```

Open it in VS Code.

At the top right, select the kernel:

```
Python (Deep Learning)
```

Then run cells normally.

Test cell

```
import torch
import numpy as np

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

20. Everyday Workflow for Students

Each time you start work:

Step 1

Open VS Code in:

```
Documents\student_id
```

Step 2

Open a terminal

Step 3

Run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\activate
```

Step 4

Check the interpreter if needed

Step 5

Open the desired project folder and run scripts or notebooks

21. How to Add More Packages Later

If a future project needs another package, install it into the same environment:

```
python -m pip install package_name
```

Example:

```
python -m pip install plotly
```

22. How to Save the Installed Environment

To export the exact installed packages:

```
python -m pip freeze > .\deepLearningCourse\requirements-exported.txt
```

This creates a snapshot of the current environment.

23. Common Problems and Fixes

Problem: `pip install ...` is blocked

Use: `python -m pip install ...`

Problem: activation script is blocked

Run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

then:

```
.\venv\Scripts\activate
```

Problem: wrong Python is being used

Run:

```
python -c "import sys; print(sys.executable)"
```

If it does not point to `venv\Scripts\python.exe`, reselect the interpreter in VS Code.

Problem: GPU install says “No matching distribution found”

This usually means one of these:

- wrong Python/package combination
- old version pins in the requirements file
- wrong index URL
- CPU PyTorch already installed

For current Windows GPU installs, use the official CUDA 12.8 wheel index format from PyTorch’s install page. ([PyTorch](#))

Problem: `torch.cuda.is_available()` is False

Possible reasons:

- CPU build of PyTorch is installed
- wrong CUDA wheel was used
- outdated NVIDIA driver
- unsupported GPU or driver setup

24. Setup Summary - Recommended Commands

CPU setup

```
python -m venv venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\activate
python -m pip install --upgrade pip setuptools wheel
```

```
python -m pip install -r .\deepLearningCourse\requirements-cpu.txt
python -m pip install torch torchvision torchaudio
python -m ipykernel install --user --name dl_env --display-name "Python
(Deep Learning)"
```

GPU setup

```
python -m venv venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r .\deepLearningCourse\requirements-gpu-
cu128.txt
python -m pip install -r .\deepLearningCourse\requirements-gpu-
rest.txt
python -m ipykernel install --user --name dl_env --display-name "Python
(Deep Learning)"
python -c "import torch; print(torch.__version__);
print(torch.version.cuda); print(torch.cuda.is_available())"
```